

Exercise: Understanding and Practicing Diffusion Models

Instructor: Prof. Jun Zhu

TA: Mikhail Kuvakin

Requirements:

- We recommend that you typeset your homework using appropriate software such as $\text{\LaTeX}$. If you submit your handwritten version, please make sure it is clean written up and legible. The TAs will not invest undue effort to decrypt bad handwriting.
- Submit your code and report to mkekwy25@mails.tsinghua.edu.cn. Please include both solutions and execution results in the report.

Introduction

These exercises will help you understand and implement a basic Denoising Diffusion Probabilistic Model (DDPM). Diffusion models are a class of generative models that learn to create data by reversing a gradual noising process. This problem set will guide you through the key theoretical derivations and their corresponding implementations. All code exercises are designed to be completed in the accompanying Python file, `diffusion_exercises.py`.

Recommended reference papers:

- [Denoising Diffusion Probabilistic Models](#)
- [Score-Based Generative Modeling through Stochastic Differential Equations](#)

Preliminaries

This section provides a brief, intuitive overview of the mathematical concepts needed for the exercises.

The Gaussian (Normal) Distribution

The Gaussian distribution is fundamental to diffusion models. It's defined by a mean μ (its center) and a variance σ^2 (its spread). Its probability density function is:

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

For our purposes, three properties are crucial:

1. **The Reparameterization Trick.** To sample a value x from $\mathcal{N}(\mu, \sigma^2 \mathbf{I})$ ($\mathbf{I}$ is the identity matrix), we can first sample a standard normal variable $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ and then compute $x = \mu + \sigma\epsilon$. This is vital for training, as it allows gradients to flow back through the sampling process.
2. **Sum of Independent Gaussians.** Adding two independent Gaussian variables results in another Gaussian variable. If $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$, then $X + Y \sim \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$.
3. **Product of Gaussians (Bayesian Inference).** The product of two Gaussian probability density functions is another (unnormalized) Gaussian. This property is used in Bayesian inference, where a Gaussian prior multiplied by a Gaussian likelihood results in a Gaussian posterior.

Stochastic Differential Equations (SDEs)

An **SDE** describes a path with a deterministic trend (“drift”) plus random noise (“diffusion”). The general form of an SDE is:

$$dx_t = f(x_t, t)dt + g(t)dW$$

Here:

- dx is the infinitesimal change in the state x .
- $f(x, t)dt$ is the **drift term**, representing the deterministic part of the motion.
- $g(t)dW$ is the **diffusion term**, representing the random part. dW is an infinitesimal step of a Wiener process (standard Brownian motion). When $\Delta t \rightarrow 0$, we have $\Delta t \rightarrow dt$ and $\epsilon \sqrt{\Delta t} \rightarrow dW$, where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ represents a standard random Gaussian noise.

For a small time step $t \rightarrow t + \Delta t$, we know $f(x, t)$, $g(t)$ are almost unchanged, and we approximately have the time discretization: $x_{t+\Delta t} - x_t \approx f(x_t, t) \cdot \Delta t + g(t) \cdot \epsilon \sqrt{\Delta t}$. It becomes an SDE in the limit as $\Delta t \rightarrow 0$.

1 The Forward Process

The forward process, q , gradually adds Gaussian noise to the clean data x_0 over T timesteps. The distribution of x_t at timestep t given x_{t-1} is a Gaussian:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I})$$

where β_t is a small positive constant (chosen beforehand as a hyperparameter). Letting $\alpha_t = 1 - \beta_t$ and using the reparameterization trick, we can sample x_t as: $x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1}$, where $\epsilon_{t-1} \sim \mathcal{N}(0, \mathbf{I})$.

Problem 1 (Closed-Form Forward Process). A key property of the forward process is that we can sample x_t directly from x_0 without iterating through all intermediate steps. Show that $q(x_t|x_0)$ is also a Gaussian distribution, and derive its mean and variance. Specifically, show that:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\bar{\epsilon}$$

where $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ and $\bar{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$. [Hint: Start by substituting the expression for x_{t-1} into the equation for x_t . Remember that a weighted sum of independent Gaussian variables is also a Gaussian, where variances add.]

Problem 2 (Implementing the Forward Process). Fill in the blank in the `q_sample` function in `diffusion_exercises.py` to implement the closed-form sampling of x_t given x_0 .

```
1 def q_sample(x_0, t, noise=None):
2     # ... (setup code) ...
3     # Get the \bar{\alpha}_t values for the given timesteps t
4     alphas_t = get_alphas_cumprod_at_t(t)
5
6     # YOUR CODE HERE: calculate x_t
7     noisy_x = "???"
8
9     return noisy_x
```

Listing 1: Code for Problem 1.2

2 The Reverse Process

The goal is to learn the reverse process $p_\theta(x_{t-1}|x_t)$ parameterized by our neural network θ . To do this, we first analyze the true posterior $q(x_{t-1}|x_t, x_0)$ when we know the clean data x_0 , which will inform our model’s objective.

Problem 3 (Reverse Process Posterior). The reverse posterior $q(x_{t-1}|x_t, x_0)$ is tractable and is also a Gaussian. Derive the mean and variance of this distribution. If we formulate $q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t\mathbf{I})$, prove that

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

[Hint: Use Bayes’ theorem, $q(x_{t-1}|x_t, x_0) \propto q(x_t|x_{t-1})q(x_{t-1}|x_0)$. Work in the log domain and complete the square with respect to x_{t-1} to find the posterior’s mean and variance.]

Problem 4 (The Neural Network’s Goal). The mean $\tilde{\mu}_t(x_t, x_0)$ depends on x_0 , which is unknown during sampling. We can train the network to predict the clean data x_0 directly. However, this approach does not work well in practice. Alternatively, we usually train the network to predict the noise ϵ . Prove that the posterior mean $\tilde{\mu}_t$ can also be expressed by x_t and ϵ :

$$\tilde{\mu}_t(x_t, \epsilon) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon \right)$$

[Hint: Express x_0 in terms of x_t and ϵ using the result from Problem 1.1. Substitute this expression for x_0 into the equation for $\tilde{\mu}_t(x_t, x_0)$ to show that the mean can also be expressed as a function of x_t and ϵ .]

This result shows that the entire reverse process can be found if we can predict the noise ϵ from x_t and t .

3 Implementation: Training and Sampling

From the previous section, we know the core task is to train a network $\epsilon_\theta(x_t, t)$ to predict the noise ϵ given the noised data x_t and timestep t . The simplified loss function is:

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\|\epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2 \right]$$

Once trained, we can use the model to reverse the process, starting from pure noise $x_T \sim \mathcal{N}(0, \mathbf{I})$ and sampling $x_{t-1} \sim p_\theta(x_{t-1}|x_t)$ iteratively, where the reverse step $p_\theta(x_{t-1}|x_t)$ is obtained by replacing the true noise ϵ in the true posterior $\mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t \mathbf{I})$ with the network-predicted noise ϵ_θ .

Problem 5 (The Training Loop). Fill in the blanks in the code in `diffusion_exercises.py` to complete the training loop.

```

1 # ... (inside the training loop) ...
2 # 3. Sample random noise from standard Gaussian distribution
3 # YOUR CODE HERE
4 noise = "???"
5
6 # 4. Create the noisy sample x_t using the q_sample function
7 # YOUR CODE HERE
8 x_t = "???"
9
10 # ... (model prediction) ...
11
12 # 6. Calculate the loss with the MSE (mean square error) function
13 # YOUR CODE HERE
14 loss = "???"

```

Listing 2: Code for Problem 3.1

Problem 6 (The Sampling Loop). Fill in the blanks in the `p_sample_loop` function in `diffusion_exercises.py`. This involves using the trained model ϵ_θ to calculate the mean of the reverse step and then drawing a sample.

```

1 @torch.no_grad()
2 def p_sample_loop(model, shape):
3     # ... (setup code, loop from T-1 down to 0) ...
4     # ... (predict noise, get parameters) ...
5
6     # The mean of the distribution for x_{t-1}
7     # YOUR CODE HERE
8     mean = "???"
9
10    # Get the variance and add noise
11    if t_val > 0:
12        alpha_t_cumprod_prev = alphas_cumprod[t_val - 1]
13
14        # The variance of the distribution for x_{t-1}
15        # YOUR CODE HERE
16        posterior_variance = "???"
17
18    # ... (add variance noise) ...
19    return x

```

Listing 3: Code for Problem 3.2

4 The Continuous-Time Limit (Optional)

The discrete-time diffusion process can be seen as an approximation of a continuous-time process described by a Stochastic Differential Equation (SDE), providing a deeper connection to score-based modeling.

Problem 7 (Forward and Reverse SDEs). Consider the discrete forward update $x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_{t-1}$. Let time be continuous, such that $\beta_t = \beta(t)\Delta t$ for a small step Δt .

1. Derive the forward SDE of the form $dx = f(x, t)dt + g(t)dW$ and express $f(x, t)$ and $g(t)$ with $\beta(t)$. [Hint: Use the approximation $\sqrt{1 - u} \approx 1 - u/2$ for small u .]
2. The reverse of this SDE is given by $dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)]dt + g(t)d\bar{W}$, where $\nabla_x \log p_t(x)$ is the score function. Show that our noise prediction network $\epsilon_\theta(x_t, t)$ is learning a quantity proportional to this score function.